SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CONCEPT MAPPING

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Concept Mapping
Weightage:	20% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)		
Student Name:	R.NANDHINI	Reg. No.:	192524355
Section / Batch:		Date:	

Instructions to Students

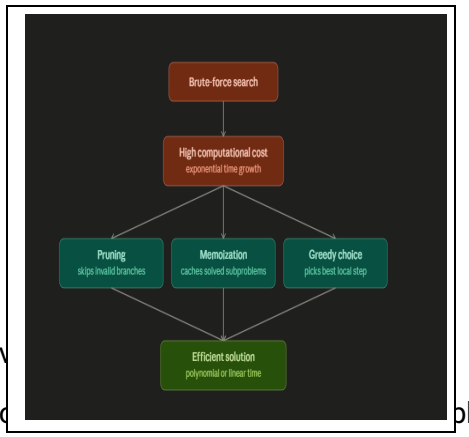
- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Draw a clear and neat concept map for each question.
- Identify all key concepts and arrange them in a logical hierarchy.
- Show meaningful linkages between concepts using arrows and connecting words.
- Compare related concepts wherever required and justify the relationships clearly.
- Ensure the concept map is well-organized, readable, and properly labelled.

Question Type	Focus Area	Questions	Marks
Concept Mapping	Map algorithm efficiency concepts using asymptotic analysis, search techniques, recurrence relations, and recursion tree method	Q1 – Q2	Q1 – 50 Q2 – 50
	GRAND TOTAL:	100 Marks	

SECTION A – CONCEPT MAPPING

Q1. Space Complexity of Graph Traversal Algorithms Analyze the space complexity of BFS and DFS algorithms. Clearly define data structures used. Apply asymptotic notation to represent memory usage. Use diagrams to illustrate queue and stack behavior. Present results and interpret efficiency.

Answer:



Why brute force gets expensiv

A brute-force algorithm sc ble option and checking each one against the goal. It doesn't reason about which paths are promising — it just enumerates. That "try everything" strategy is what drives the cost up:

- Search space grows combinatorially. If each decision point has k choices and there are n decision points, brute force explores roughly k^n possibilities. Adding one more input element doesn't add a little work — it multiplies the total work.
- No reuse of prior work. Overlapping subproblems (like recomputing Fibonacci(5) many times inside a recursive tree) get solved from scratch every time they reappear, wasting cycles on identical calculations.
- No early rejection. Even paths that are obviously doomed (a partial sum that already exceeds the target, a board state that's already invalid) get carried forward and fully explored anyway.

The result is time complexity like $O(2^n)$ or $O(n!)$ — fine for tiny inputs, unusable once n grows past a few dozen.

How each optimization technique cuts that cost

Pruning — stops exploring a branch as soon as it can be proven that branch cannot lead to a valid or better solution (e.g., backtracking with constraint checks, alpha-beta pruning in game trees). It doesn't change what the correct answer is; it just refuses to waste time confirming failure paths that are already implied by the rules. This shrinks the effective search tree, often turning exponential blowup into something tractable in practice, even though the worst case is still exponential.

Memoization — stores the result of each subproblem the first time it's computed and reuses that stored value on later encounters, instead of recomputing it. This directly targets the "overlapping subproblems" source of waste. Fibonacci is the classic case: naive recursion is $O(2^n)$, but caching each Fibonacci(k) collapses it to $O(n)$, because each unique subproblem is now solved exactly once.

Greedy choice — at each step, commits to the option that looks best right now, without backtracking or exploring alternatives at all. This eliminates branching entirely for problems where a greedy strategy is

provably optimal (minimum spanning tree via Kruskal/Prim, Huffman coding, activity selection), taking complexity down to O(n log n) or O(n). The trade-off: greedy only works when the problem has the right structure (optimal substructure + greedy-choice property) — used on the wrong problem, it gives a fast but wrong answer.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established

Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Marks Evaluation:

Question No.	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)	Total Marks (Each – 50)
Q1						
Q2						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____